Commande des moteurs avec Arduino

Objectifs du TP

- Comprendre le rôle des actionneurs, et en particulier des moteurs, dans un système robotique.
- Commander un moteur à courant continu dans les deux sens à l'aide de deux modules relais.
- Ajouter une commande manuelle par bouton-poussoir (gestion de l'anti-rebond).
- Comprendre le principe du signal PWM et la commande d'un servomoteur en position.
- Asservir la position d'un servomoteur à un potentiomètre.

Matériel nécessaire

Composant	Quantité	Remarque
Carte Arduino UNO + câble USB	1	
Module relais 1 voie (5 V)	2	Entrées IN, VCC, GND ; contacts NO, COM, NC
Moteur à courant continu	1	Avec son alimentation externe (+V)
Bouton-poussoir	1	Exercice 2
Servomoteur SG90	1	Exercices 3 et 4
Potentiomètre 10 kΩ	1	Exercice 4
Plaque d'essai + fils de connexion	1 lot	

Partie 1 : Moteur à courant continu et module relais

1.1 Les actionneurs dans un système robotique

Un système robotique est organisé autour de trois fonctions : percevoir (capteurs), décider (unité de traitement, ici l'Arduino) et agir (actionneurs). Un actionneur transforme un ordre, généralement électrique, en action physique : mouvement, force, chaleur, lumière, son.

Les moteurs sont les actionneurs les plus répandus en robotique, car ce sont eux qui produisent le mouvement : rotation des roues d'un robot mobile, déplacement des articulations d'un bras robotique, orientation d'une caméra, ouverture d'une pince, etc. On distingue principalement :

- **Le moteur à courant continu (DC) :** simple et peu coûteux, il tourne tant qu'il est alimenté. Sa vitesse dépend de la tension, son sens dépend de la polarité. Il est adapté à la propulsion (roues, ventilateurs).
- **Le servomoteur :** moteur associé à un réducteur, un capteur de position et une électronique de contrôle. Il se place à un angle précis. Il est adapté aux articulations.
- **Le moteur pas à pas :** il tourne par pas angulaires fixes, pour des positionnements précis sans capteur (imprimantes 3D, CNC).

1.2 Pourquoi un module relais ?

Le moteur DC ne possède pas de broche de commande : il possède seulement deux bornes d'alimentation. De plus, une sortie numérique de l'Arduino délivre 5 V sous un courant limité (environ 20 mA recommandés, 40 mA maximum), ce qui est très insuffisant pour faire tourner un moteur. Il faut donc un organe intermédiaire qui permette à un petit signal de commander un circuit de puissance : c'est le rôle du relais.

Un relais est un interrupteur commandé électriquement. Une bobine, alimentée via un transistor présent sur le module, attire un contact mécanique. Le circuit de commande (Arduino) et le circuit de puissance (moteur) sont ainsi séparés électriquement. Le bornier de puissance comporte trois contacts :

- **COM** (commun) : contact mobile.
- **NC** (Normally Closed, normalement fermé) : relié à COM quand le relais est au repos.
- **NO** (Normally Open, normalement ouvert) : relié à COM quand le relais est activé.

> **Remarque : modules actifs à l'état bas**
>
> Beaucoup de modules relais du commerce sont « actifs LOW » : le relais colle quand l'entrée IN est à 0 V. Vérifiez le comportement de votre module (LED témoin) avant de commencer, et inversez HIGH/LOW dans le code si nécessaire.

1.3 Inverser le sens de rotation avec deux relais

Le sens de rotation d'un moteur DC dépend du sens du courant qui le traverse. Avec deux relais, on relie chaque borne du moteur (A et B) au COM d'un relais ; les contacts NC sont reliés au +V et les contacts NO à la masse. On obtient ainsi un pont en H électromécanique.

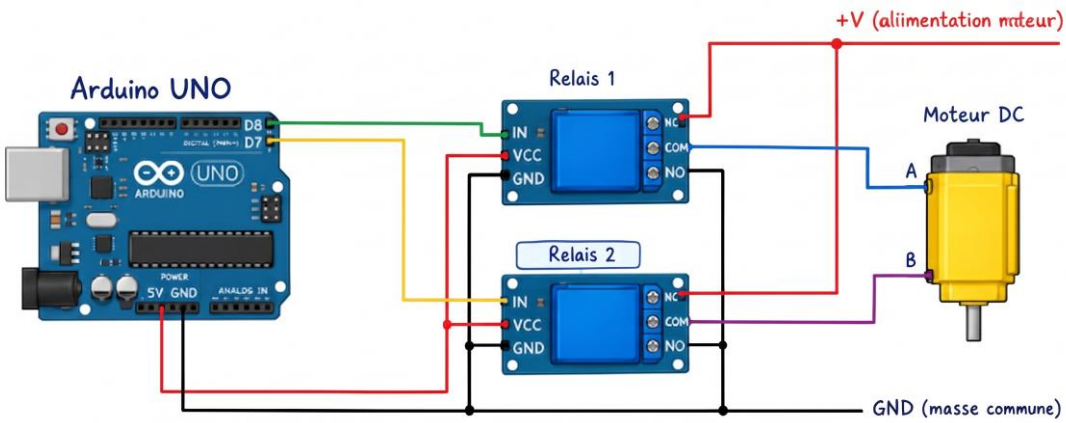


Figure 1 : Commande d'un moteur DC dans les deux sens avec deux relais

Relais 1	Relais 2	Borne A	Borne B	État du moteur
Repos	Repos	+V	+V	Arrêt (freinage)
Activé	Repos	GND	+V	Rotation sens 1
Repos	Activé	+V	GND	Rotation sens 2
Activé	Activé	GND	GND	Arrêt (freinage)

Remarque : aucune combinaison ne met le +V en court-circuit avec la masse. Ce câblage est donc sûr, même en cas d'erreur de programmation.

Exercice 1 : Commande d'un moteur dans les deux sens

Réaliser le montage de la figure 1 : IN du relais 1 sur D8, IN du relais 2 sur D7, VCC sur 5 V et GND sur GND de l'Arduino. La masse de l'alimentation du moteur doit être commune avec celle de l'Arduino. Vérifiez que les numéros de broches du programme correspondent à votre câblage.

```
// Définition des broches de commande reliées aux entrées du module relais
const int pinIN1 = 7;
const int pinIN2 = 8;
```

```
void setup() {
  // Configuration des broches en sorties numériques
  pinMode(pinIN1, OUTPUT);
  pinMode(pinIN2, OUTPUT);

  // Initialisation : Mettre les deux relais au repos (arrêt du moteur)
  // Note : Selon le type de module relais (actif LOW ou HIGH),
  // ajustez digitalWrite à HIGH ou LOW si nécessaire.
  digitalWrite(pinIN1, LOW);
  digitalWrite(pinIN2, LOW);
}

void loop() {
  // --- SENS 1 ---
  // Relais 1 activé, Relais 2 au repos
  digitalWrite(pinIN1, HIGH);
  digitalWrite(pinIN2, LOW);
  delay(2000); // Le moteur tourne dans le sens 1 pendant 2 secondes

  // --- ARRÊT ---
  // Les deux relais au repos (les deux bornes du moteur sont au +V)
  digitalWrite(pinIN1, LOW);
  digitalWrite(pinIN2, LOW);
  delay(1000); // Pause d'une 1 seconde

  // --- SENS 2 ---
  // Relais 1 au repos, Relais 2 activé
  digitalWrite(pinIN1, LOW);
  digitalWrite(pinIN2, HIGH);
  delay(2000); // Le moteur tourne dans le sens 2 pendant 2 secondes

  // --- ARRÊT ---
  digitalWrite(pinIN1, LOW);
  digitalWrite(pinIN2, LOW);
  delay(1000); // Pause d'une 1 seconde
}
```

Questions

1. Téléverser le programme et décrire le comportement du moteur.
2. Compléter le tableau de la section 1.3 en observant l'état des LED des modules relais.
3. Pourquoi prévoit-on une phase d'arrêt entre les deux sens de rotation ?
4. Modifier le programme pour que le moteur tourne 3 s dans le sens 1 puis 1 s dans le sens 2.

Exercice 2 : Commande manuelle par bouton-poussoir

On ajoute un bouton-poussoir entre la broche D9 et la masse. Grâce à la résistance de tirage interne (INPUT_PULLUP), la broche lit HIGH au repos et LOW quand le bouton est appuyé. Chaque appui fait passer le moteur à l'état suivant : Arrêt → Sens 1 → Sens 2 → Arrêt.

Bouton	Connexion
Patte 1	Broche D9 de l'Arduino
Patte 2	GND

Anti-rebond : un bouton mécanique ne commute pas proprement ; son contact « rebondit » pendant quelques millisecondes et génère plusieurs transitions. Le programme n'accepte une nouvelle lecture que si elle est stable depuis 50 ms (debounceDelay).

```
// Relay control pins
```

```
const int pinIN1 = 7;
const int pinIN2 = 8;

// Push button pin
const int buttonPin = 9;

// Variables for button handling and states
int motorState = 0; // 0: Stop, 1: Forward, 2: Reverse
bool lastButtonState = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long debounceDelay = 50; // Debounce time in milliseconds

void setup() {
  // Initialize relay pins as outputs
  pinMode(pinIN1, OUTPUT);
  pinMode(pinIN2, OUTPUT);

  // Initialize button pin with internal pull-up resistor
  pinMode(buttonPin, INPUT_PULLUP);

  // Start with the motor stopped
  stopMotor();
}

void loop() {
  bool reading = digitalRead(buttonPin);

  // Check for button press with debouncing
  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }

  if ((millis() - lastDebounceTime) > debounceDelay) {
    static bool previousReading = HIGH;

    // Detect button press (transition from HIGH to LOW because of INPUT_PULLUP)
    if (reading == LOW && previousReading == HIGH) {
      // Cycle through states: 0 -> 1 -> 2 -> 0
      motorState = (motorState + 1) % 3;

      // Apply action based on the current state
      switch (motorState) {
        case 0:
          stopMotor();
          break;
        case 1:
          moveForward();
          break;
        case 2:
          moveReverse();
          break;
      }
    }
    previousReading = reading;
  }
  lastButtonState = reading;
}

// --- Helper Functions ---
void stopMotor() {
  digitalWrite(pinIN1, LOW);
  digitalWrite(pinIN2, LOW);
}

void moveForward() {
```

```
  digitalWrite(pinIN1, HIGH);
  digitalWrite(pinIN2, LOW);
}

void moveReverse() {
  digitalWrite(pinIN1, LOW);
  digitalWrite(pinIN2, HIGH);
}
```

Point d'attention : inversion directe du sens

Dans ce programme, le passage de l'état 1 (sens 1) à l'état 2 (sens 2) se fait sans arrêt intermédiaire. Le moteur, encore lancé, reçoit brutalement une tension inverse : pic de courant, contraintes mécaniques et usure des contacts des relais. Une solution consiste à passer par l'arrêt avant chaque inversion :

```
void moveReverse() {
  stopMotor();      // on passe d'abord par l'arrêt
  delay(300);       // le moteur ralentit avant l'inversion
  digitalWrite(pinIN1, LOW);
  digitalWrite(pinIN2, HIGH);
}
```

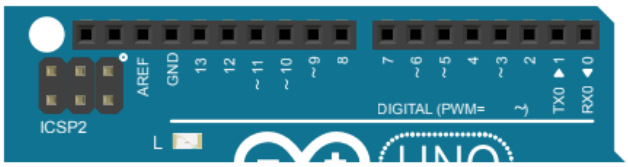
Questions

1. Pourquoi utilise-t-on INPUT_PULLUP au lieu d'une résistance externe ?
2. Que se passe-t-il si on met debounceDelay à 0 ? Tester et expliquer.
3. Intégrer la correction ci-dessus dans moveForward() et moveReverse().
4. Ajouter un second bouton : un bouton pour le sens, un bouton pour marche/arrêt.

Partie 2 : Signal PWM et servomoteur

2.1 Le signal PWM

Une sortie analogique est une source de tension pilotable. La carte Arduino ne possède pas de vraie sortie analogique, capable de produire une tension d'une valeur quelconque choisie par l'utilisateur. Certains ports numériques peuvent cependant simuler une sortie analogique en utilisant la technique de PWM (Pulse Width Modulation, ou modulation de largeur d'impulsions) : il s'agit des ports 3, 5, 6, 9, 10, et 11 (signalés par un ~ sur la carte).



Ces ports peuvent basculer rapidement entre leur état LOW (0 V) et HIGH (5 V). En contrôlant le rapport du temps passé dans l'état HIGH par rapport au temps passé dans l'état LOW, la carte Arduino peut varier la valeur moyenne de la tension sur ces ports entre 0 et 5 volts. Le schéma ci-dessous illustre ce principe en représentant la variation au cours du temps de la tension du port 3 utilisé en sortie PWM pour différentes tensions demandées.

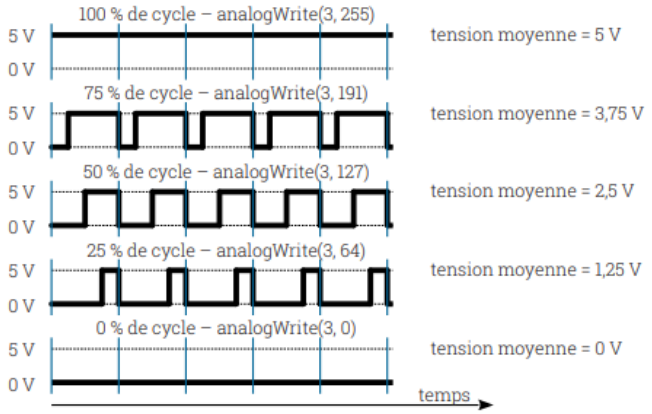


Figure 2 : Principe de la modulation de largeur d'impulsion (PWM) sur Arduino

La fonction analogWrite(broche, valeur) génère sur les broches marquées ~ (3, 5, 6, 9, 10, 11) un signal carré d'environ 490 Hz (980 Hz sur les broches 5 et 6), dont le rapport cyclique est proportionnel à la valeur (0 à 255). Ce type de PWM sert à faire varier une tension moyenne : luminosité d'une LED, vitesse d'un moteur DC via un transistor ou un pont en H.

2.2 Le servomoteur

En robotique, beaucoup de mouvements doivent être précis et reproductibles : articulations d'un bras robotique, pince, direction d'un véhicule, orientation d'un capteur ou d'une caméra, gouvernes d'un drone ou d'un avion. Le servomoteur répond à ce besoin : il intègre un moteur DC, un réducteur, un potentiomètre qui mesure la position de l'axe et une carte électronique qui corrige en permanence l'écart entre la position demandée et la position mesurée. C'est un système asservi en boucle fermée.



Figure 3 : Servomoteur SG90 et son brochage

Fil du servo	Rôle	Connexion Arduino
Rouge	Alimentation +5 V	5 V
Marron	Masse	GND
Orange	Signal de commande	D9

2.3 Commande d'un servomoteur : un PWM particulier

Le servomoteur n'est pas commandé par la tension moyenne comme à la figure 2. Il attend une impulsion toutes les 20 ms (50 Hz), et c'est la durée de cette impulsion qui code l'angle : environ 1 ms pour 0°, 1,5 ms pour 90° et 2 ms pour 180° (les valeurs exactes varient selon les modèles, typiquement 0,5 à 2,5 ms pour le SG90).

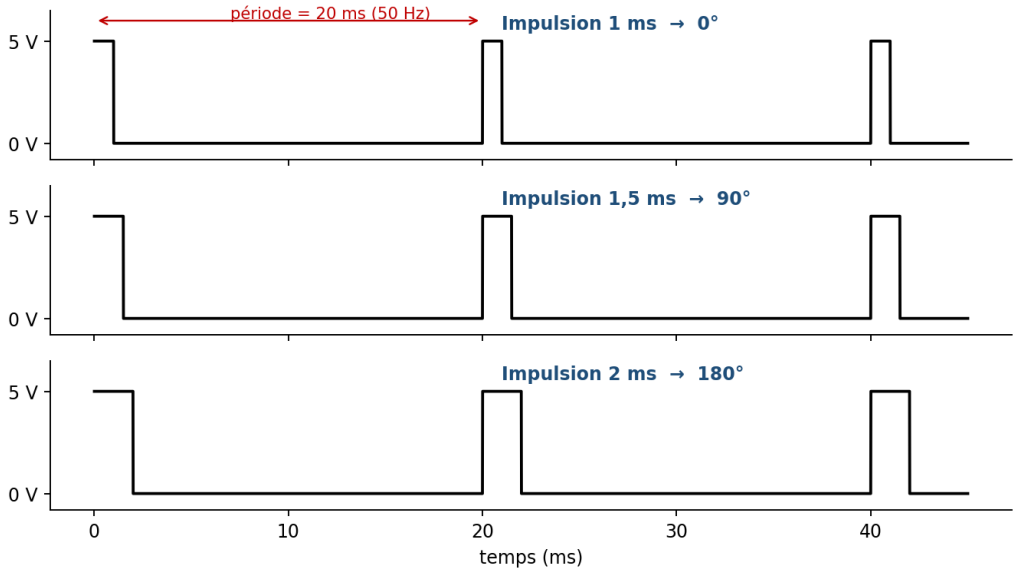


Figure 4 : Signal de commande d'un servomoteur (période 20 ms)

> **À retenir**
>
> On n'utilise pas analogWrite() pour un servomoteur : sa fréquence (≈ 490 Hz) ne correspond pas aux 50 Hz attendus. On utilise la bibliothèque Servo, qui génère les bonnes impulsions : monServo.write(angle) avec un angle entre 0 et 180.

Exercice 3 : Positionner un servomoteur à des angles précis

Programme 3-a : positions fixes.

```
#include <Servo.h>

// Création de l'objet servo
Servo monServo;

// Définition de la broche de commande du servo
const int servoPin = 9;

void setup() {
  // Attache le servomoteur à la broche numérique 9
  monServo.attach(servoPin);
}

void loop() {
  // 1. Positionner le moteur à 0 degré
  monServo.write(0);
  delay(1000); // Attend 1 seconde

  // 2. Positionner le moteur à 90 degrés (position centrale)
  monServo.write(90);
  delay(1000); // Attend 1 seconde

  // 3. Positionner le moteur à 180 degrés
  monServo.write(180);
  delay(1000); // Attend 1 seconde

  // 4. Aller à 45 degrés
  monServo.write(45);
  delay(1000);
}
```

Programme 3-b : balayage progressif de 0° à 180°.

```
#include <Servo.h>

Servo monServo;
const int servoPin = 9;

void setup() {
  monServo.attach(servoPin);
}

void loop() {
  // Aller de 0° à 180° doucement
  for (int angle = 0; angle <= 180; angle += 1) {
    monServo.write(angle);
    delay(15); // Vitesse du mouvement (plus la valeur est grande, plus c'est lent)
  }

  // Revenir de 180° à 0° doucement
  for (int angle = 180; angle >= 0; angle -= 1) {
    monServo.write(angle);
    delay(15);
  }

  delay(1000); // Pause d'une seconde avant de recommencer
}
```

Questions

1. Quelle différence de mouvement observez-vous entre les programmes 3-a et 3-b ?
2. Dans le programme 3-b, calculer la durée d'un aller 0° → 180°.
3. Modifier le programme 3-b pour que l'aller soit rapide et le retour lent.
4. Le servo atteint-il réellement 0° et 180° ? Tester monServo.attach(9, 500, 2400) et comparer.

Exercice 4 : Commande du servomoteur par potentiomètre

On souhaite maintenant que l'angle du servomoteur suive la position d'un potentiomètre, comme un joystick qui pilote une articulation de bras robotique. Le potentiomètre fonctionne en diviseur de tension : son curseur délivre une tension entre 0 et 5 V, que l'Arduino convertit en une valeur entre 0 et 1023 avec analogRead().

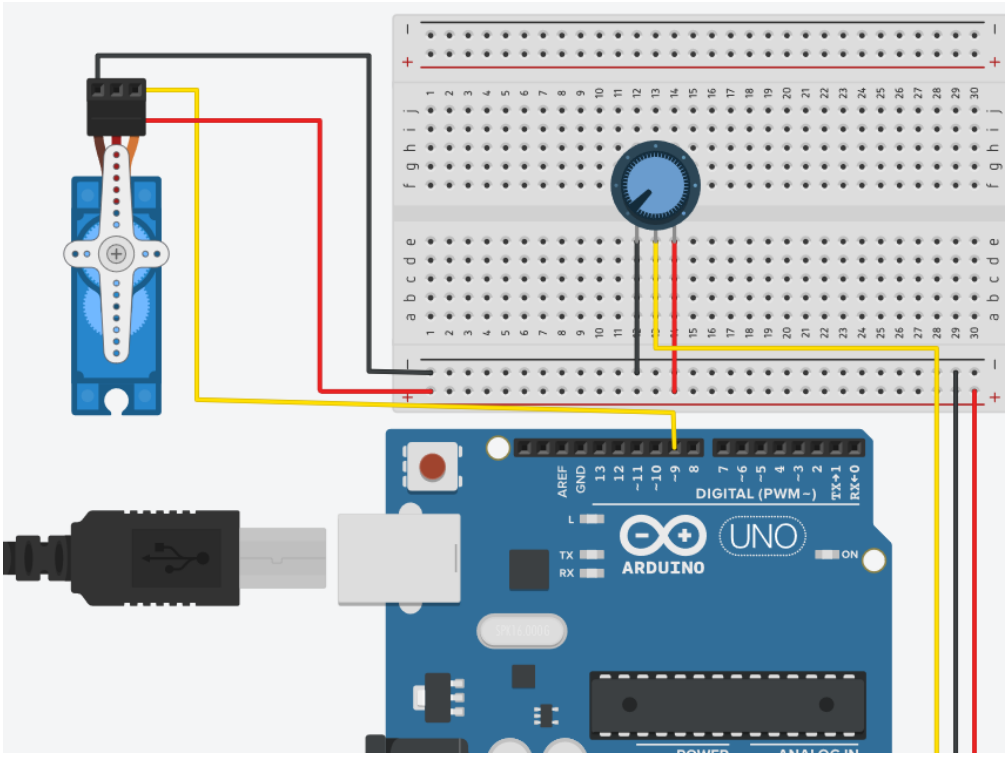


Figure 5 : Commande d'un servomoteur par un potentiomètre

Composant	Broche	Connexion
Potentiomètre	Extrémité 1	GND
Potentiomètre	Curseur (centre)	A0
Potentiomètre	Extrémité 2	5 V
Servo	Orange (signal)	D9
Servo	Rouge / Marron	5 V / GND

La fonction map() convertit linéairement la plage 0–1023 en une plage 0–180°.

```
#include <Servo.h>

// Création de l'objet servo
Servo monServo;

// Broches utilisées
const int servoPin = 9;   // Signal du servo (fil orange)
const int potPin   = A0;  // Curseur (broche centrale) du potentiomètre

void setup() {
```

```
  monServo.attach(servoPin);
  Serial.begin(9600);     // Pour afficher les valeurs dans le moniteur série
}

void loop() {
  // 1. Lecture du potentiomètre : valeur entre 0 et 1023 (0 V à 5 V)
  int valeurPot = analogRead(potPin);

  // 2. Conversion de la plage 0-1023 vers la plage d'angle 0-180°
  int angle = map(valeurPot, 0, 1023, 0, 180);

  // 3. Commande du servomoteur
  monServo.write(angle);

  // 4. Affichage pour vérification
  Serial.print("Potentiometre = ");
  Serial.print(valeurPot);
  Serial.print("  ->  Angle = ");
  Serial.println(angle);

  delay(15); // Laisse au servo le temps d'atteindre la position
}
```

Questions

1. Ouvrir le moniteur série (9600 bauds) et relever les valeurs pour trois positions du potentiomètre.
2. Calculer l'angle obtenu pour une valeur lue de 512. Vérifier.
3. Limiter le mouvement du servo entre 30° et 150° (butées d'une articulation).
4. Le servo tremble parfois au repos. Proposer une solution logicielle (moyenne de plusieurs lectures, ou mise à jour seulement si l'angle change de plus de 2°).

Remarque sur l'alimentation

Un servo SG90 peut consommer plusieurs centaines de mA au démarrage ou en blocage. Alimenté par le 5 V de l'Arduino via USB, il peut provoquer des redémarrages de la carte. Pour plusieurs servos, ou un servo plus gros, utiliser une alimentation 5 V externe avec la masse commune à l'Arduino, et ajouter un condensateur (100 à 470 µF) sur l'alimentation du servo.